

ECTiny集成说明文档

目录

- 1. 前言
 - 1.1. 简介
 - 1.2. ECTiny 架构图
 - 1.3. ECTiny 设计说明
 - 1.4. 工作内容
- 2. ECTiny 使用方法
 - 2.1. 基础模块
 - 2.1.1. 注册激活
 - 2.1.1.1. 必要条件
 - 2.1.1.2. 激活方案
 - 2.1.1.3. 激活方法
 - 2.1.1.4. 证书保存
 - 2.1.2. 使用流程图
 - 2.1.2.1. WIFI状态通知说明
 - 2.1.2.2. 初始化相关参数说明（ ECConfigHandle ）
 - 2.1.3. 初始化代码示例
 - 2.1.3.1. 集成代码示例
 - 2.2. BLE 蓝牙组网模块
 - 2.2.1. 模块说明
 - 2.2.2 工作内容
 - 2.2.3 厂商BLE服务规范
 - 2.2.3.1 厂商BLE广播数据规范
 - 2.2.3.1.1 Service UUID
 - 2.2.3.1.2 Service Data
 - 2.2.3.1.3 广播数据实例
 - 2.2.3.2 厂商BLE服务规范
 - 2.2.3.2.1 Device Infomation(0x180A)
 - 2.2.3.3 亿连BLE服务(0000b360-d6d8-c7ec-bdf0-eab1bfc6bcbcb)
 - 2.2.3 连接流程图
 - 2.2.4 接口说明
 - 2.2.4.1 蓝牙读写数据接口
 - 2.2.4.2 蓝牙数据回调接口
 - 2.2.4.3 蓝牙组网主动调用接口
 - 2.2.4.4 接口调用时序图
 - 2.3. Mirror 模块
 - 2.3.1. 使用流程图
 - 2.3.2. 代码示例
 - 2.3. Audio 模块
 - 2.4.1. 使用流程图
 - 2.4.2. 代码示例
 - 2.5. 反控/键控/快捷方式
 - 2.6. 手机APP交互模块
 - 2.6.1. 代码示例
 - 2.7. OTA 模块
 - 2.8. 工具模块
- 3. ECTiny HMI开发帮助
 - 3.1. 亿连HMI标准页面
 - 3.2. ECTiny HMI接口交互流程

- [3.2.1. 连接建立流程](#)
- [3.2.2. 投屏流程](#)

修订记录

版本/状态	起始日期	ECTiny版本	作者	备注
V1.0	2020-4-3	V1.0	Tao	初稿
V1.0	2020-6-3	V1.0	Like	初稿

1. 前言

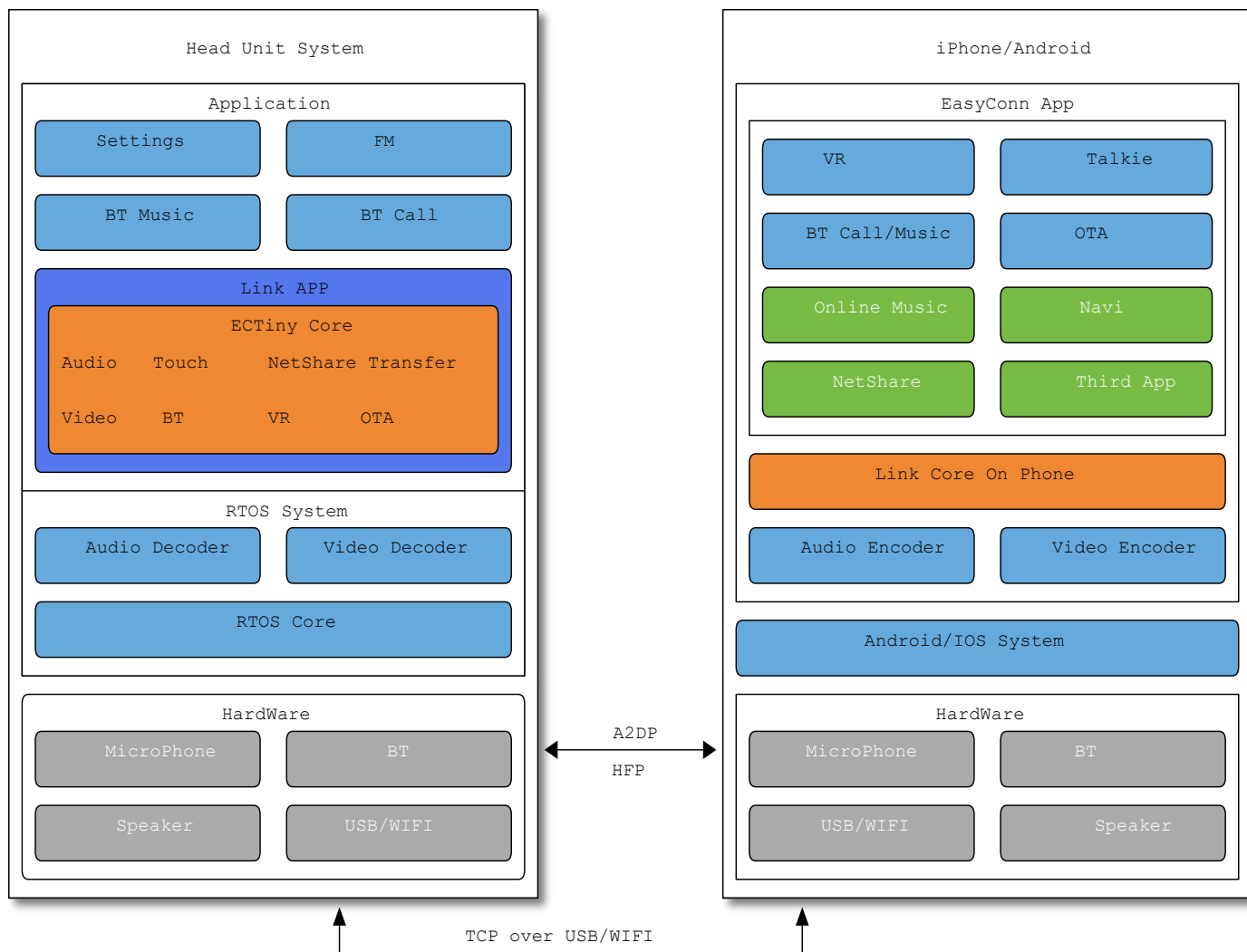
1.1. 简介

ECTiny 是由武汉卡比特信息有限公司，开发用于 eCos 、 FreeRTOS 、 Linux 等操作系统平台的手机互联开发包，主要采用C 语言开发，可移植性强。

区别于**ECSDK**手机互联开发包（另外一套手机互联产品），**ECSDK**目前主要适用于Linux、Android、QNX操作系统，主要采用C++语言开发，支持功能更加全面。**ECTiny** 是一套精简版的手机互联产品。

如果需要对ECTiny工作原理、互联技术更深入的了解，以及互联产品软硬件要求，ECTiny合作分工、ECTiny项目验收流程等，请参考《亿连手机互联SDK开发指南》。

1.2. ECTiny 架构图



1.3. ECTiny 设计说明

ECTiny 采用模块化的设计，主要划分了8个模块：

- **基础模块**

基础模块，主要提供ECTiny的运行、释放、版本获取，以及互联程序的相关配置选项等。

- **Mirror模块**

视频模块，提供视频相关的配置，解码、渲染，视频通道状态通知等接口。

- **Audio模块**

音频模块，提供音频相关的接口，包括录音、播放、配置、交互等；

- **手机APP交互模块**

手机APP管理模块，主要提供接收APP的相关状态，以及HUD、音乐meta等接口；

- **触控/键控模块**

触控/键控模块，提供视频反控，键控等接口；

- **OTA模块**

OTA模块，提供OTA的相关接口。 **暂不支持。**

- **工具模块**

互联工具模块，提供了时间同步，GPS同步等功能。

1.4. 工作内容

模块	亿连工作	厂商工作
运行环境	构建手机互联软件环境	构建手机互联硬件和系统环境
前期工作	手机端互联服务和应用App	提供车机唯一ID
^	移植ECTiny	^
连接	ECTiny基础模块提供与手机建立通讯链路	查看文档 2.1，使用基础模块与手机App建立基础通讯
证书	连接过程中自动完成证书校验	当证书校验失败时，给予用户页面展示
投屏	提供Mirror功能模块	查看文档2.2，实现解码渲染功能，监听投屏消息
音频	提供Audio功能模块	查看文档2.3，实现音频播放，录音数据上传
键控/反控	提供触控功能模块	查看文档2.4，开发功能快捷方式，反控
APP交互	提供App功能模块	查看文档2.5，开发与App交互的定制功能
OTA	提供OTA功能模块	查看文档2.6，利用OTA开发在线升级软件功能
页面交互	提供相关的连接/投屏等状态	参考 3 章节

2. ECTiny 使用方法

2.1. 基础模块

此模块主要完成ECTiny初始化、启动、USB读写设备绑定等工作。

- 使用
 - 实现 IECCallback 定义的相关回调函数；
 - 初始化 EC_initialize；
 - 运行 EC_start；
 - 绑定USB设备 EC_bindUSBDevice、EC_bindHidDevice；
 - 停止 EC_stop；

2.1.1. 注册激活

- 为保证优质的产品质量和售后服务，卡比特针对亿连手机互联产品提供了一套完整认证授权机制,激活成功的车机，最终会生成一个license文件保存在车机本地。如果车机由于升级而造成的文件丢失，需要车机重新通过手机互联，并保证手机联网重新下载证书。

2.1.1.1. 必要条件

- 提供车机硬件唯一ID标识。
- 保证车机硬件ID在车机重置、重启、升级等情况下，都不会变更。

2.1.1.2. 激活方案

- 方案1：标准激活
厂商提供备案Android手机IMEI和激活数量给亿连，由卡比特客户经理后台备案。在生成阶段，需要使用联网备案Android手机逐个激活车机并下载证书。
- 方案2：批量激活
厂商提供所有待生成车机的ID号给亿连(推荐连续ID范围)，由卡比特客户经理后台批量备案即可完成激活。若有漏激活车机，可采用方案1补充。

2.1.1.3. 激活方法

车机通过ECSDK接口首次传入车机ID时，ECSDK会通过手机APP验证ID的合法性。如果验证失败，则ECSDK会终止当前互联流程。如果激活成功，则会
下载授权证书，并保存到车机本地。以后只要本地证书存在且有效，则车机端互联不会再走激活的流程。

2.1.1.4. 证书保存

鉴于大多数小系统不支持本地文件读写，为此ECTiny将文件IO抽象成接口，由客户实现将证书数据保存到车机本地，并且车机重启、升级等都不会销毁本地证书文件内容。

- 示例

```
static uint32_t size(const char* name)
{
    //返回name对应存储区域的数据大小
    return length;
}

static int32_t read(const char* name,void *data, uint32_t length)
{
    //将name对应的数据拷贝到data缓冲区
    if(length > bufferSize)
    {
        return 0;
    }
    memcpy(data, buffer, length);
    return length;
}

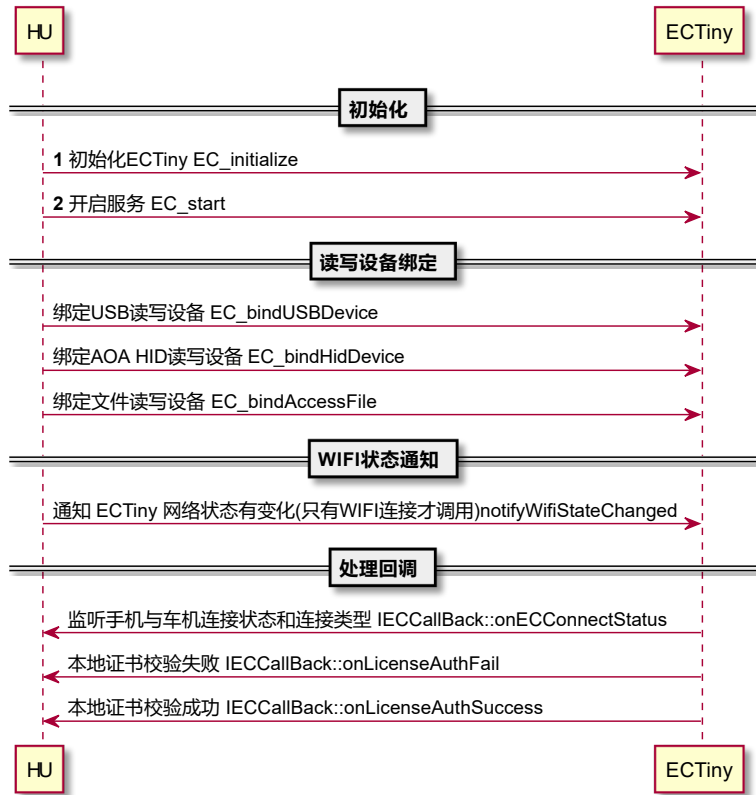
static int32_t write(const char* name,void *data, uint32_t length)
{
    //将data缓冲区数据拷贝到name对应的存储区域
    if(length > sizeof(buffer))
    {
        return 0;
    }
    memcpy(buffer, data, length);
    bufferSize = length;
    return length;
}

static void clear(const char* name)
{
    //销毁name对应的存储区域
    memset(buffer, 0, sizeof(buffer));
    bufferSize = 0;
}

static IECAccessFile license2File = {
    .size = size,
    .read = read,
    .write = write,
    .clear = clear
};
```

2.1.2. 使用流程图

ECTiny 基础模块



HMI交互流程开发，参考 3.2.1 章节。

2.1.2.1. WIFI状态通知说明

- ECTiny 是通过发布mdns发现同一局域网的互联设备，并且 ECTiny 不参与车机系统和手机之间的组网过程，所以需要车机通知 ECTiny 此时的网络状态，以便 ECTiny 可以正确的发布mdns。

ps:此接口有且只能调用一次当车机网络状态已经发生改变。

- 接口说明

接口	参数	参数描述	是否必须
notifyWifiStateChanged	ECWifiStateAction action	EC_WIFI_STATE_CHANGED_ACTION : AP,STATION 模式 EC_WIFI_P2P_STATE_CHANGED_ACTION : P2P, wlan直连	Y
^	ECNetWorkInfo::state	P2P和STATION 有四种状态，分别是可用、不可用、连接、断开连接 比如 state = EC_WIFI_STATE_ENABLE 或者 state = EC_WIFI_STATE_CONNECTED 。	Y
^	^	AP只有两种状态即可用、不可用。	Y
^	ECNetWorkInfo::phoneIp	连接的手机ip地址，如获取不到可以不填。	O
^	ECNetWorkInfo::carIp	车机的ip地址，如获取不到可以不填。	O

2.1.2.2. 初始化相关参数说明（ECConfigHandle）

- 该参数主要对ECTiny的相关授权功能进行配置

接口	参数	参数描述	是否必须
----	----	------	------

接口	参数	参数描述	是否必须
ECSDKConfig(const string& uuid, const string& version, const string& writableDir)	uuid	车机uuid	Y
^	version	互联软件版本号	Y
^	writableDir	车机的可写目录，用于保存临时文件及证书 (由于小系统大多数不支持文件读写，所以此参数传空即可)	N
setCommonConfig	cfgName	额外的参数配置, 每个参数都有一个唯一的标识	N
^	cfgValue	该参数的值	^

- 针对额外的参数，参考ECSDK使用指南6.2.2章节，下面举例说明：
 - ECAuthentication参数配置，key是"auth_" 加上参数名，比如 key="auth_flavor"
 - ECCarDescription参数配置，key是"car_" 加上参数名，比如 key="car_supportConnect"
 - ECOptions参数配置，key是"opt_"加上参数名，比如 key="opt_bluetoothPolicy"

Y：必须设置项; N:SDK合作项目使用不到，无需关心;O：可选项;

2.1.3. 初始化代码示例

```
1 //设置日志等级
2 EC_setLogInfo(EC_LOG_LEVEL_ALL,EC_LOG_OUT_STD,NULL,EC_LOG_MODULE_SDK);
3
4 // 设置sdk初始化参数
5 ECConfigHandle cfg = EC_createECConfig();
6 EC_setBaseConfig(cfg, "CARBIT0000001","1.0",NULL);
7
8 // IECCallBack 一般由ui继承，实现基类的纯虚函数，以显示不同的界面和提示
9 IECCallBack listener;
10 EC_initialize(cfg, &listener);
```

2.1.3.1. 集成代码示例

```

1 void IECCallback::onECConnectStatus(ECConnectedStatus status, ECConnectedType type)
2 {
3     ECLOGD("[ECSdkListener]:onSdkConnectType status=%d,type=%d",status,type);
4
5     // todo 获取连接类型
6     // 可以根据不同的连接类型，给予不同的界面提示或者界面跳转
7     switch (type)
8     {
9         // todo
10        case EC_CONNECT_TYPE_ANDROID_USB:
11            break;
12        case EC_CONNECT_TYPE_ANDROID_WIFI:
13            break;
14        case EC_CONNECT_TYPE_IOS_USB_EAP:
15            break;
16        case EC_CONNECT_TYPE_IOS_USB_MUX:
17            break;
18        case EC_CONNECT_TYPE_IOS_USB_AIRPLAY:
19            break;
20        case EC_CONNECT_TYPE_IOS_WIFI_APP:
21            break;
22        case EC_CONNECT_TYPE_IOS_WIFI_AIRPLAY:
23            break;
24        case EC_CONNECT_TYPE_IOS_USB_LIGHTNING:
25            break;
26        default:
27            break;
28    }
29
30    // todo 获取连接状态
31    // 根据不同的连接状态，车机界面给予用户不同的提示或者界面跳转
32    switch (status)
33    {
34        case EC_CONNECT_STATUS_DEVICE_ATTACHED:
35        {
36            // todo 设备接入
37            break;
38        }
39        case EC_CONNECT_STATUS_DEVICE_DEATTACHED:
40        {
41            // todo 设备拔除
42            break;
43        }
44        case EC_CONNECT_STATUS_CONNECTING:
45        {
46            // todo 正在连接
47            break;
48        }
49        case EC_CONNECT_STATUS_CONNECT_FAILED:
50        {
51            // todo 连接失败
52            break;
53        }
54        case EC_CONNECT_STATUS_CONNECT_SUCCEED:
55        {
56            // todo 连接成功
57            break;
58        }
59        case EC_CONNECT_STATUS_DISCONNECTED:
60        {
61            // todo 连接断开
62            break;
63        }
64        case EC_CONNECT_STATUS_APP_EXIT:
65        {
66            // todo 手机app退出
67            break;
68        }
69        case EC_CONNECT_STATUS_INTERRUPTED_BY_APP:
70        {

```



```
71         // todo 手机app中断
72         break;
73     }
74     default:
75         break;
76 }
77 }
78
79 void IECCallback::onLicenseAuthFail(int32_t errCode, const string& errMsg)
80 {
81     ECLOGD("[ECSDKListener]:onLicenseAuthFail errCode=%d,errMsg=%s",errCode,errMsg.c_str());
82
83     // todo 授权失败。此回调被执行时，SDK会停止工作。需要提示联系供应商。
84     //
85 }
86 void IECCallback::onLicenseAuthSuccess(int code, const string& msg)
87 {
88     ECLOGD("[ECSDKListener]:onLicenseAuthSuccess code=%d,msg=%s",code,msg.c_str());
89
90     // todo 授权成功。
91     //
92 }
```

2.2. BLE 蓝牙组网模块

2.2.1. 模块说明

蓝牙BLE互联互通协议是由卡比特公司开发的手机和车机通过蓝牙BLE互联协议。它采用c语言开发，采用协议开源和源代码开源的方式提供合作方开发。

开发包主要提供：

2.2.2 工作内容

模块	亿连工作	厂商工作
运行环境	无	带有蓝牙BLE服务的系统环境
前期工作	提供渠道id，亿连BLE服务UUID，读写特征UUID	按照亿连提供规范写入BLE广播数据

2.2.3 厂商BLE服务规范

2.2.3.1 厂商BLE广播数据规范

2.2.3.1.1 Service UUID

采用16bit service uuid方式。其值固定为0x6967。

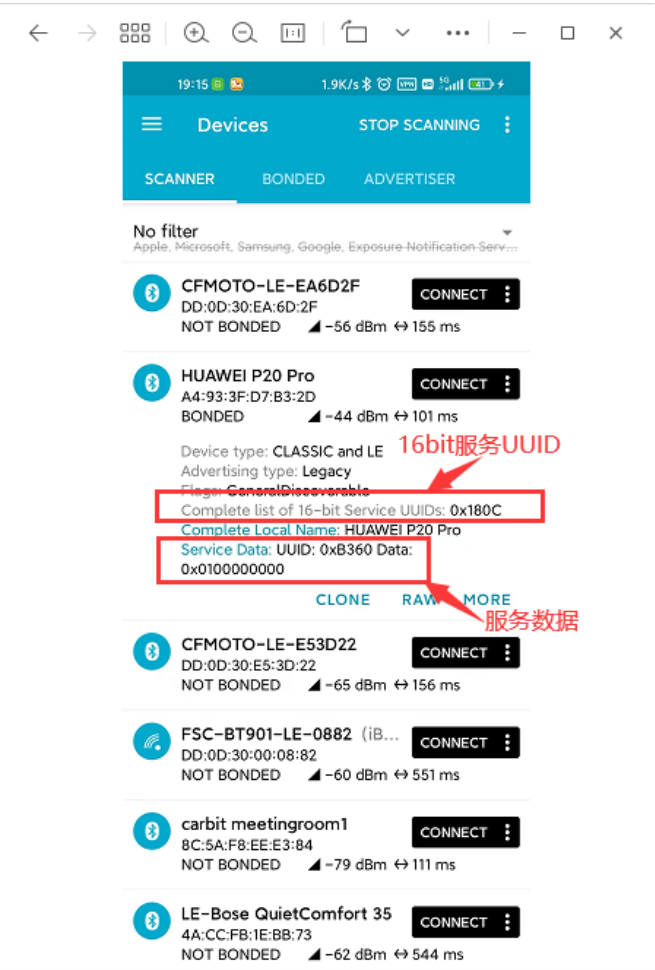
2.2.3.1.2 Service Data

服务数据UUID为16bit uuid，其值是固定值。通过SDK接口 get16bitServiceDataUUID() 获得数据的长度为5个字节。通过SDK接口 getBLEServiceData() 获得。

2.2.3.1.3 广播数据实例

LEN	TYPE	VALUE	备注
2	0x01	0x02	flag，厂商自定
3	0x03	0x0C18	16bit 服务uuid，由亿连提供
15	0x09	0x485541574549205032302050726F	蓝牙名称，产商自定

LEN	TYPE	VALUE	备注
8	0x16	0x60B30100000000	服务数据，通过SDK接口获取

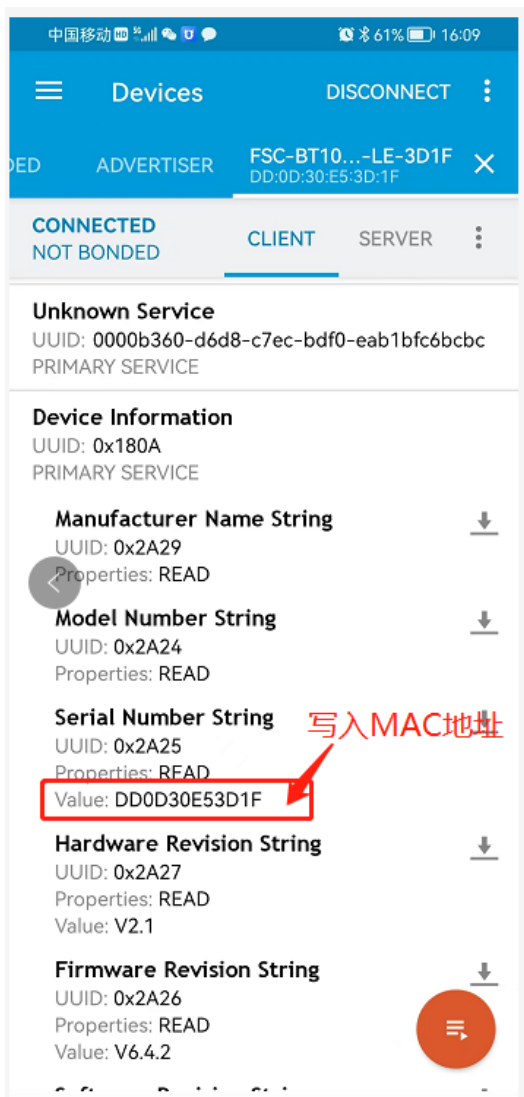


2.2.3.2 厂商BLE服务规范

2.2.3.2.1 Device Infomation(0x180A)

厂商需要添加标准Device Infomation服务UUID(0x180A), 并且将特征Serial Number String(0x2A25)的值填写为蓝牙MAC地址。

示例：



2.2.3.3 亿连BLE服务(0000b360-d6d8-c7ec-bdf0-eab1bfc6bcbc)

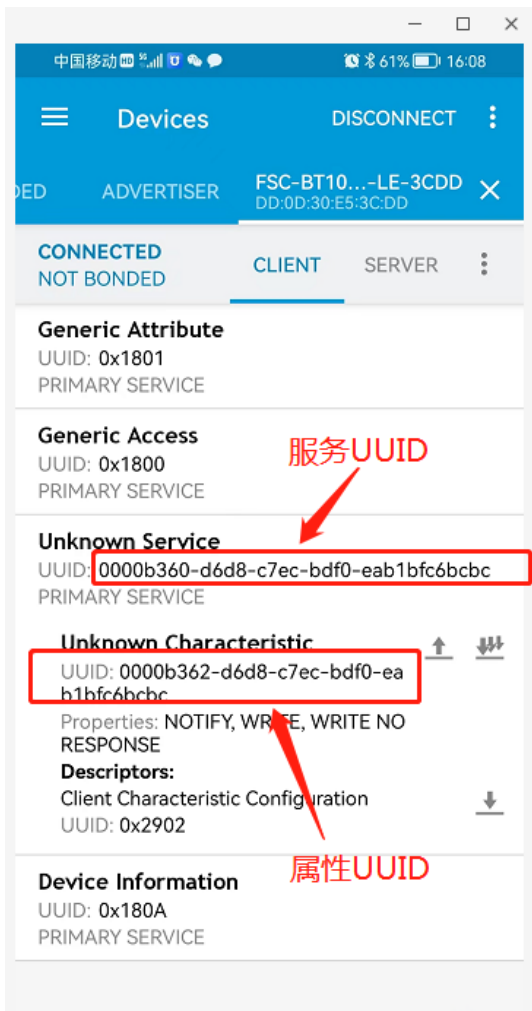
服务UUID0000b360-d6d8-c7ec-bdf0-eab1bfc6bcbc

读写特征:0000b362-d6d8-c7ec-bdf0-eab1bfc6bcbc

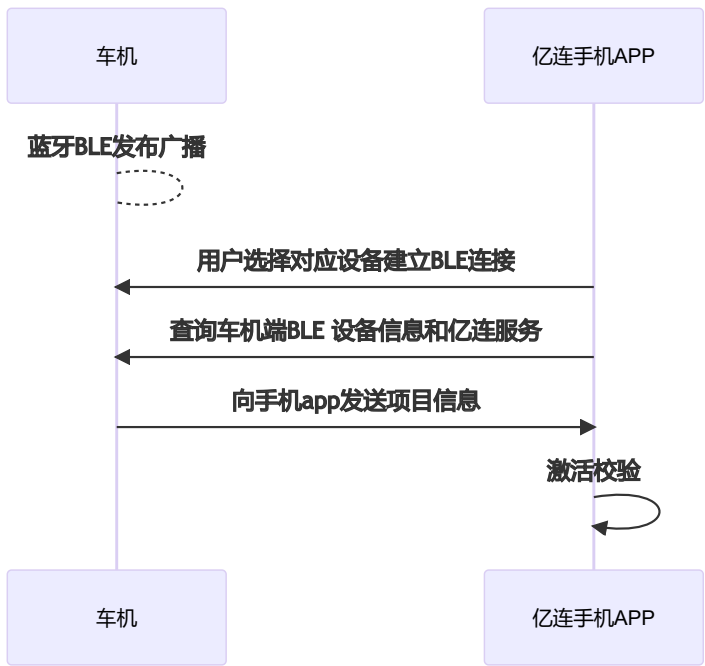
厂商需要添加亿连蓝牙BLE通讯服务UUID(), 并且添加读写特征。

读写特征的需要添加NOTIFY, WRITE, WRITE NO RESPONSE等属性

示例:



2.2.3 连接流程图



2.2.4 接口说明

2.2.4.1 蓝牙读写数据接口

- 蓝牙读写数据接口

```

1 | typedef struct
2 | {
3 |     int32_t (*read)(uint8_t* data, uint32_t len);
4 |     int32_t (*write)(const uint8_t* data, uint32_t len);
5 |     int32_t (*interrupt)();
6 | }IECBTAccessDev;

```

仪表/车机端必须实现此接口。

此接口用于读取、写入、中断蓝牙数据。

此接口通过 EC_bindBTDevice 方法注册给互联。

互联通过此接口收发蓝牙数据，与手机进行通讯。

2.2.4.2 蓝牙数据回调接口

- 蓝牙数据回调接口

```

1 | typedef struct {
2 |     ...
3 |     void (*onRequestBuildNet)(const ECBTClientInfo *clientInfo);
4 |     void (*onRequestBuildNetCancel)();
5 |     void (*onPhoneBuildNetFinish)();
6 |     void (*onPhoneAPIInfo)(const ECBTNetInfo* netDeviceInfo);
7 |     ...
8 | } IECCallBack;

```

仪表/车机端必须实现此接口。

手机app通过蓝牙发送给仪表/车机的数据，会通过此接口回调给仪表/车机端的程序。

- IECCallBack::onRequestBuildNet 手机端发送的组网请求，车机端在此回调执行时需要调用 EC_setRequestBuildNetRly 通知给手机将会以何种方式组网；
- IECCallBack::onRequestBuildNetCancel 手机端通知车机端取消组网；
- IECCallBack::onPhoneBuildNetFinish 手机端通知车机端组网完成；
- IECCallBack::onPhoneAPIInfo 手机端通知车机端，此时手机作为ap的相关信息；

2.2.4.3 蓝牙组网主动调用接口

- 主动调用接口

```

1 | int32_t EC_bindBTDevice( IECBTAccessDev* ioHandle);
2 | int32_t EC_unBindBTDevice();
3 | int32_t EC_notifyBLEStateChanged(const ECBTConnectStatus status);
4 | int32_t EC_setRequestBuildNetRly(const char *phoneID, const ECBTRequestBuildNetRly *rly);
5 | int32_t EC_setNetInterfaceInfo(const ECNetInterfaceInfo *info, const int32_t num);

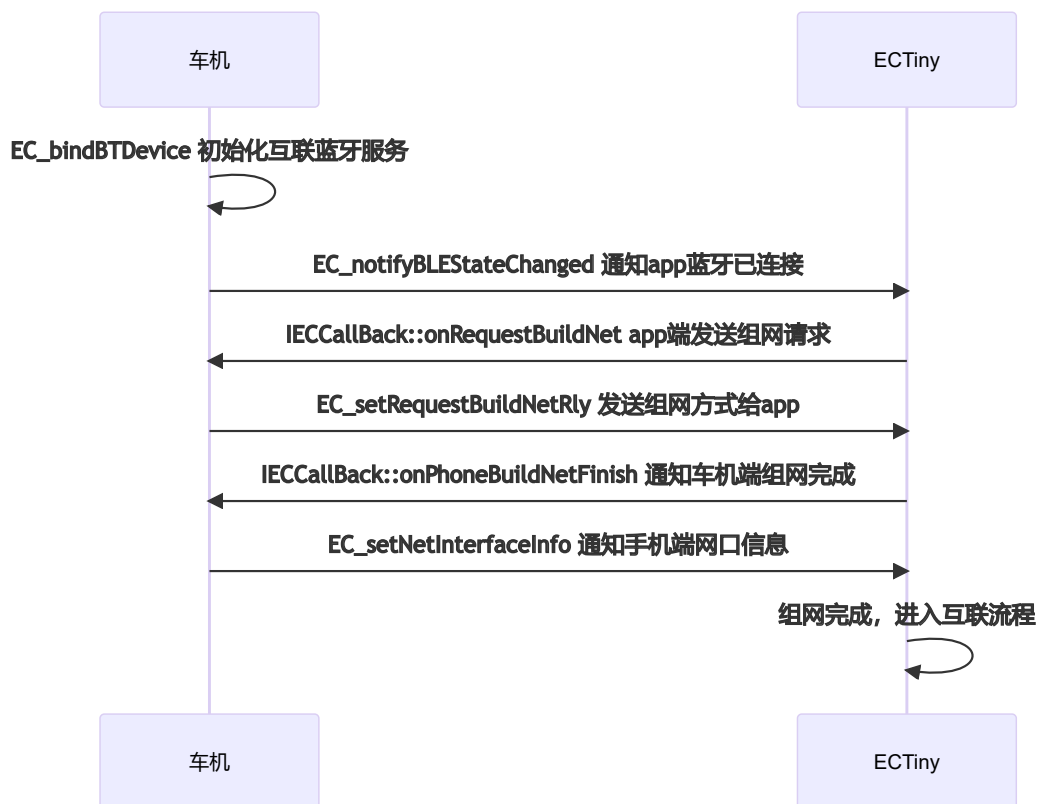
```

车机端程序主动调用这些方法进行BLE组网

- EC_bindBTDevice 初始化蓝牙服务，主要传入 IECBTAccessDev 实例；
- EC_unBindBTDevice 释放蓝牙服务；
- EC_notifyBLEStateChanged 通知互联，当前蓝牙连接状态；
- EC_setRequestBuildNetRly 给互联发送组网信息，通知互联将以何种方式组网；
- EC_setNetInterfaceInfo 给互联发送网络接口信息，互联会自动连接网络；

2.2.4.4 接口调用时序图

- 此时序图用以说明蓝牙组网接口的调用时序



2.3. Mirror 模块

该模块主要实现互联投屏的相关功能，主要包括设置视频播放设备、设置投屏参数、开启/关闭视频。

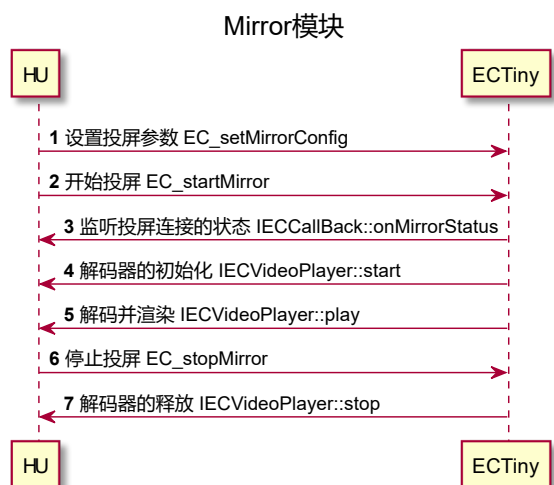
• 注意事项

此模块的所有功能都是基于 `IECCallBack::onECConnectStatus::EC_CONNECT_STATUS_CONNECT_SUCCEED` 的状态，即车机 ECTiny 与手机设备传输链路建立成功。

• 使用

- 根据 `IECVideoPlayer` 定义的 `play` , `stop` , `play` 接口，实现解码器设备的解码、渲染功能。
- 一般情况互联程序在前台时，如果需要投屏，此时需要主动调用 `EC_startMirror` , 在后台时调用 `EC_stopMirror` , 根据 `IECCallBack::onMirrorStatus` 来判断是否调用成功。

2.3.1. 使用流程图



2.3.2. 代码示例

- 集成 IECCallback::onMirrorStatus 关键代码

```
1  /*
2  * 用户实现：投屏状态监听类
3  * onMirrorStatus: sdk framework 会通过 status 把投屏状态通过此回调函数传递出来。用户根据需要处理不同的状态
4  * */
5  void IECCallback::onMirrorStatus(ECMirrorStatus status)
6  {
7      ECLOGD("[IECCallback]:onMirrorStatus status=%d",status);
8
9      switch (status)
10     {
11         case EC_MIRROR_STATUS_MIRROR_AUTH_PENDING:
12         {
13             // todo 投屏授权未通过。
14             // 建议在车机界面上提示用户在手机上点击同意授权
15             break;
16         }
17         case EC_MIRROR_STATUS_MIRROR_STARTED:
18         {
19             // todo 投屏成功
20             //
21             break;
22         }
23         case EC_MIRROR_STATUS_MIRROR_FAILED:
24         {
25             // todo 投屏失败
26             // 建议车机界面提示连接失败
27             break;
28         }
29         case EC_MIRROR_STATUS_MIRROR_STOPPED:
30         {
31             // todo 停止投屏操作成功
32             //
33             break;
34         }
35         default:
36             break;
37     }
38 }
39
```

- 集成VideoPlayer关键代码

```

1      /*
2      * 用户实现：视频播放器功能
3      * start：初始化并启动播放器
4      * stop：停止并释放播放器
5      * play：播放手机传输过来的视频数据
6      */
7      class VideoPlayer : public IECVideoPlayer
8      {
9      public:
10         VideoPlayer();
11         ~VideoPlayer();
12
13         virtual void play(const void *data, int32_t len) override;
14         virtual void start(int width, int height) override;
15         virtual void stop() override;
16
17     private:
18         std::atomic_bool mHasInit;
19         std::mutex      mMutex;
20     };
21
22     void VideoPlayer::play(const void *data, int32_t len)
23     {
24         lock_guard<mutex> lock(mMutex);
25         if(!mHasInit)
26             return;
27
28         // todo 视频数据传输给解码器
29
30     }
31
32     void VideoPlayer::start(int width, int height)
33     {
34         lock_guard<mutex> lock(mMutex);
35         if(mHasInit) return;
36
37         // todo 初始化指定宽高渲染窗口
38
39         // todo 初始化解码器
40
41         mHasInit = true;
42     }
43
44     void VideoPlayer::stop()
45     {
46         ECLOGD("[VideoPlayer]:stop_begin");
47         lock_guard<mutex> lock(mMutex);
48         if (!mHasInit)
49             return;
50
51         // todo 释放解码器
52
53         // todo 释放渲染窗口
54
55         mHasInit = false;
56         ECLOGD("[VideoPlayer]:stop_end");
57     }

```

有关投屏连接的建立和原理，请详见《亿连手机互联SDK开发指南》6.4 章节。

2.3. Audio 模块

该模块主要实现互联音频的相关功能，主要包括设置音频播放设备、设置录音设备、开启/关闭音频等。

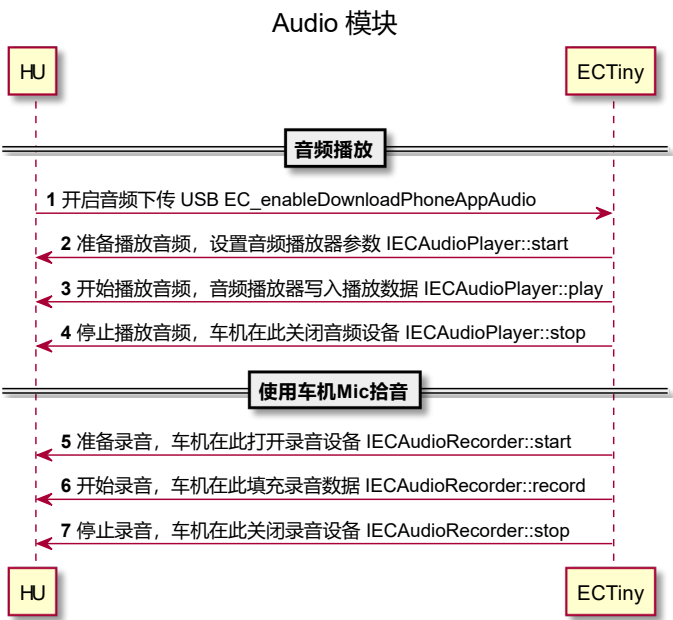
• 注意事项

此模块的所有功能都是基于 `IECCallBack::onECConnectStatus::EC_CONNECT_STATUS_CONNECT_SUCCEED` 的状态，即车机 ECTiny 与手机设备传

输链路建立成功。

- 使用
 - 调用 EC_enableDownloadPhoneAppAudio 确定音源需要传输的音频类型；
 - 实现 IECAudioPlayer 所有接口，实现音频播放器的初始化、播放、释放等功能。
 - 继承 IECAudioRecorder 并实现所有接口，实现录音设备的初始化，填充录音数据，释放等功能。

2.4.1. 使用流程图



2.4.2. 代码示例

- 集成 IECAudioPlayer 关键代码

```

1  /*
2  * 用户实现：声音播放器功能
3  * start：初始化并启动播放器
4  * stop：停止并释放播放器
5  * play：播放手机传输过来的音频数据
6  * setVolume：设置音量
7  */
8  typedef struct
9  {
10     void    (*start)(ECAudioType type, const ECAudioInfo *info);
11     void    (*stop)(ECAudioType type);
12     void    (*play)(ECAudioType type, const void *data, uint32_t len);
13     void    (*setVolume)(ECAudioType type, uint32_t vol);
14 }IECAudioPlayer;
15
16 void AudioPlayer::start(ECAudioType type, const ECAudioInfo* info)
17 {
18     ECLOGD("[AudioPlayer]:start type=%d", type);
19
20     // todo 初始化声音播放器。
21     // 不同类型的声音可能会使用不同的采样率。
22     // 相同类型的声音也可能使用不同的采样率。
23     // 比如 EC_AUDIO_TYPE_TALKIE 类型的声音，采样率可能发生变化。
24     // 需要播放完成后关闭声音通道，并在下一次start时，按照传入的采样率重新打开声音通道
25
26 }
27
28 void AudioPlayer::stop(ECAudioType type)
29 {
30     ECLOGD("[AudioPlayer]:stop type=%d", type);
31
32     // todo 关闭声音播放器
33 }
34
35 void AudioPlayer::play(ECAudioType type, const void* data, uint32_t len)
36 {
37     // todo 向车机的播放器传输数据
38 }
39
40 void AudioPlayer::setVolume(ECAudioType type, uint32_t vol)
41 {
42     ECLOGD("[AudioPlayer]:setVolume type=%d", type);
43
44     // todo 设置播放器声音
45 }
46

```

- 集成IECAudioRecorder关键代码

```

1      /*
2      * 用户实现：声音录音功能
3      * 此功能使用车机的麦克风录音，并传输给手机。
4      *
5      * start：初始化录音设备
6      * record：上传录音数据
7      * stop：停止录音设备
8      *
9      */
10     typedef struct
11     {
12         int32_t (*start)(const ECAudioInfo *info);
13         void      (*stop)();
14         int32_t (*record)(void *data, uint32_t len);
15     }IECAudioRecorder;
16
17     void AudioRecorder::start(const ECAudioInfo* info)
18     {
19         ECLOGD("[AudioRecorder]:start ");
20
21         // todo 初始化录音设备
22     }
23
24     int32_t AudioRecorder::record(void *data, uint32_t len)
25     {
26         // todo 上传录音数据，将录音的数据，填充data，
27         //这个接口是由ECTiny调用的，用户在这里将录音缓冲区的数据填充到data即可
28
29         return 0;
30     }
31
32     void AudioRecorder::stop()
33     {
34         ECLOGD("[AudioRecorder]:stop");
35
36         // todo 停止录音设备
37     }

```

2.5. 反控/键控/快捷方式

此模块封装了ECSDK的投屏反控、键控功能。

• 注意事项

此模块的所有功能都是基于 IECCallBack::onECConnectStatus::EC_CONNECT_STATUS_CONNECT_SUCCEED 的状态，即车机 ECTiny 与手机设备传输链路建立成功。

• 使用

- 调用 EC_sendTouchEvent 上传触摸事件，实现反控功能，详情请参见《亿连手机互联SDK开发指南》6.6.1.2 章节。
- 调用 EC_sendBtnEvent ， 键控，详情请参见《亿连手机互联SDK开发指南》6.6.2.1.2 章节。
- 调用 EC_openAppPage ， 指定功能的快捷方式，详情请参见《亿连手机互联SDK开发指南》6.6.3 章节。

2.6. 手机APP交互模块

此模块封装了ECTiny 与手机App交互的功能，提供车机端与手机App交互的接口，包括手机App运行状态，蓝牙电话，音乐等功能的使用状态。具体接口功能请参见《亿连手机互联SDK开发指南》6.9章节。

• 注意事项

此模块的所有功能都是基于 IECCallBack::onECStatusMessage` IECCallBack::onPhoneAppHUD 等接口，监听手机App的回调消息。

• 使用

- 实现 IECCallBack::onECStatusMessage` IECCallBack::onPhoneAppHUD 等接口，监听手机App的回调消息。
- 更多功能及使用详见：《亿连手机互联SDK开发指南》6.9章节

2.6.1. 代码示例

```
1 void IECCallBack::onECStatusMessage(ECStatusMessage status)
2 {
3     ECLOGD("[APPListener]:onECStatusMessage status=%d",status);
4
5     // todo 互联状态通知消息
6     // 可以根据需求需要处理这些通知消息。具体消息内容可参看开发文档
7     switch (status)
8     {
9         // todo
10    }
11 }
12
13 void IECCallBack::onPhoneAppHUD(const ECNavigationHudInfo* data)
14 {
15     ECLOGD("[APPListener]:onPhoneAppHUD begin");
16
17     // todo 导航回调信息
18     // 由手机下发过来的导航数据。车机可以由这些数据进行导航展示。
19     // 需要实现此功能，请参考：enableDownloadPhoneAppHud 和 disableDownloadPhoneAppHud
20
21     ECLOGD("[APPListener]:onPhoneAppHUD end");
22 }
23
24 void IECCallBack::onPhoneAppInfo(const char* info)
25 {
26     ECLOGD("[APPListener]:onPhoneAppInfo begin");
27
28     // todo 手机系统信息
29     // info:目前主要以 json 的格式回调出来，目前信息量比较小，也可以自定义解析。
30     // 主要包括手机的操作系统、手机系统版本、手机型号、手机 IP 地址。
31     // 手机的 IP 地址可用于 WIFI Direct 等场景。
32
33     ECLOGD("[APPListener]:onPhoneAppInfo end");
34 }
35
36 void APPListener::onCallAction(ECCallType type, const string& name, const string& number)
37 {
38     ECLOGD("[APPListener]:onCallAction begin");
39
40     // todo 使用车机蓝牙拨打蓝牙电话功能
41     // 受限于 Android/iOS 系统的权限，无法完成电话的拨打和接听操作，因此需要依赖于车机系统的蓝牙模块完成。
42     // 当从互联内拨打电话或者挂断电话时，会通过该接口回调到车机端
43     //
44     // type: 拨打或挂断。
45     // name: 当前需要拨打或者挂断电话的名字。
46     // number: 电话号码。
47
48     ECLOGD("[APPListener]:onCallAction end");
49 }
```

2.7. OTA 模块

暂未实现，敬请期待。

2.8. 工具模块

此模块提供了一些特定的功能，如查询时间、查询天气等。

3. ECTiny HMI开发帮助

3.1. 亿连HMI标准页面

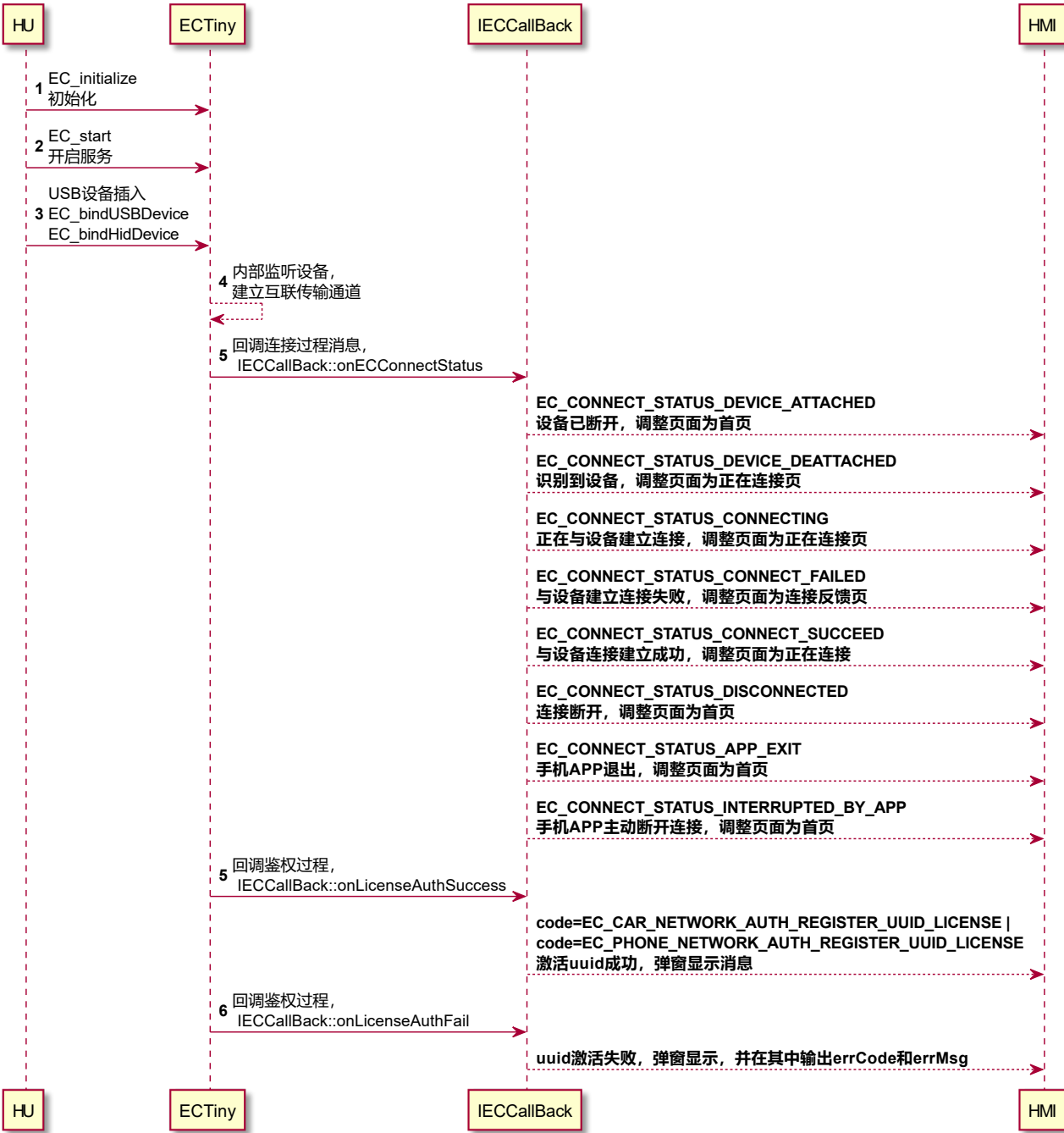
页面	描述	作用
启动页	过场页面，主要用于程序启动，或前后台切换	程序功能缓冲、增加产品露出
首页	程序默认的展示页面	等待事件触发连接开始,如等待用户插入手机
正在连接页	建立互联的过程展示页面，过程中会有相关的引导提示	程序功能缓冲、连接过程引导提示
连接反馈页	程序连接失败的展示页面	程序连接失败的引导提示
投屏页面	正在投屏的页面，手机的屏幕映射至车机系统	展示手机端的视频编码数据
静态展示页面	如关于界面、帮助页面等仅文字图片展示页面	用于展示软件相关信息

3.2. ECTiny HMI接口交互流程

ECTiny 的HMI开发，需遵循亿连的相关设计规范，具体规范请参照文档《车机端HMI设计规范说明》。以及参考《亿连手机互联SDK开发指南》 第9章节。

3.2.1. 连接建立流程

HU:车机, ECTiny:连接模块库, IECCallBack:监听连接过程消息, HMI:车机UI显示



3.2.2. 投屏流程

- 特殊说明
此功能的使用基于连接建立成功

HU:用户, ECTiny :ECTiny投屏模块, IECCallBack :监听投屏连接过程消息, HMI:车机UI显示

